



Course Name		
Computer Programming Lab		
Lab Title		
Implementing Tuple, List, and understanding aliasing, cloning		
Name	Registration Number	Lab #
Muhammad Hamzah Iqbal	F24604018	6
Date	Instructor's Name	Instructor's Signature
23/4/2024	LE Ayesha Ashfaque	

Objective:

- To have basic understanding Iteration and Recursion Functions.
- To implement Iteration and Recursion in Python.

Theoretical Aspect

- **Recursion** is a technique where a function calls itself to solve smaller parts of a problem, and it must include a base case to stop.
- **Iteration** uses loops (like for or while) to repeat code until a condition is met, making it memory-efficient and faster for most tasks.
- **Recursion vs Iteration:** Recursion is better for problems with a natural repetitive structure (like tree traversal), while iteration is preferred for performance and simplicity.
- **Fibonacci Series** is a sequence where each number is the sum of the two before it, and it can be implemented using both recursion and iteration.

Experimental Procedure

Task 01:

1. To print 'n terms of Fibonacci series using iteration. The Fibonacci numbers are the numbers in the following integer sequence. 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144.

A Fibonacci sequence is the integer sequence of 0, 1, 1, 2, 3, 5, 8.... . The first two terms are 0 and 1. All other terms are obtained by adding the preceding two terms. This means to say the nth term is the sum of (n-1)th and (n-2)th term.

Solution:

```
n = int(input("Enter the number of terms for Fibonacci series: "))
```

```
if n <= 0:
```

```
    print("Please enter a positive integer.")
```

```
else:
```

```
    a, b = 0, 1
```

```
    count = 0
```

```
    while count < n:
```

```
        print(a, end=' ')
```

```
        a, b = b, a + b
```

```
        count += 1
```

```
    print()
```

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Sun May 4 14:23:58 2025
4
5 @author: M.Hamzah Iqbal
6 """
7
8 n = int(input("Enter the number of terms for Fibonacci series: "))
9
10 if n <= 0:
11     print("Please enter a positive integer.")
12 else:
13     a, b = 0, 1
14     count = 0
15     while count < n:
16         print(a, end=' ')
17         a, b = b, a + b
18         count += 1
19     print()
```

Console 1/A X

Python 3.12.7 -- IPython 8.27.0

In [1]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/untitled0.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop')
Enter the number of terms for Fibonacci series: 6
0 1 1 2 3 5

In [2]:

Figure 1 Output

2. Write a program in C to print the Fibonacci Series using recursion.

Solution:

```
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter the number of terms for Fibonacci series: "))
if n <= 0:
    print("Please enter a positive integer.")
else:
    for i in range(n):
        print(fibonacci(i), end=' ')
    print()
```

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Sun May 4 20:46:36 2025
4
5 @author: M.Hamzah Iqbal
6 """
7
8 def fibonacci(n):
9     if n <= 1:
10         return n
11     return fibonacci(n - 1) + fibonacci(n - 2)
12
13 n = int(input("Enter the number of terms for Fibonacci series: "))
14 if n <= 0:
15     print("Please enter a positive integer.")
16 else:
17     for i in range(n):
18         print(fibonacci(i), end=' ')
19     print()
```

Console 1/A X

In [4]: runfile('C:/Users/M.Hamzah Iqbal/OneDrive/Desktop/untitled10.py', wdir='C:/Users/M.Hamzah Iqbal/OneDrive/Desktop')
Enter the number of terms for Fibonacci series: 8
0 1 1 2 3 5 8 13

In [5]:

Figure 2 Output

Discussion of Results

In this lab we achieved the following:

- Implemented recursive and iterative functions in Python.
- Calculated factorial of a number using both recursion and iteration.
- Generated Fibonacci series using iteration and recursion (in C).
- I identified the base case in recursive functions to prevent infinite calls.
- Observed performance and readability differences between iteration and recursion.

Conclusions

- Recursion breaks problems into smaller subproblems using self-calls.
- Iteration uses loops and is more memory-efficient.
- Recursive functions must have a base case to avoid infinite recursion.
- Understanding both approaches helps in choosing the right solution for a given problem.